Time Series Preprocessing Core

A Config-Driven Mini-Framework for Reproducible Preprocessing

Amy Bender, JJ Chen, Katie Harrington, Walter Hopkins, Joaquin Hoya, David Nguyen, Wei Quan, Luke Vaughan, Azton Wells, Dewen Zhong

Argonne National Laboratory

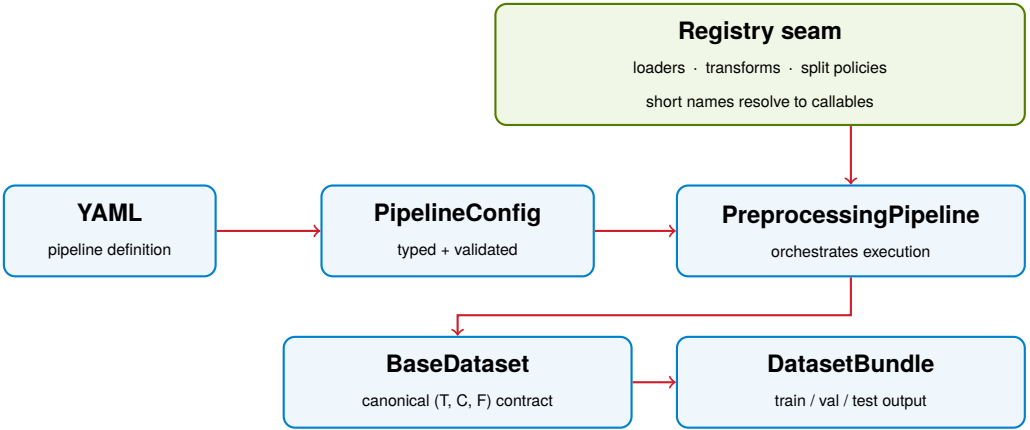
Introduction

Preprocessing-Core provides a public, reusable foundation for dataset preprocessing.

- Generic pipeline logic lives in the public core
- Private data logic stays in private repositories
- Private repos can include the core as a submodule or package
- YAML selects registered loaders, splits, and transforms
- Custom loaders and `register()` functions let each project extend the core safely

Shared preprocessing infrastructure without exposing private dataset-specific code.

Framework Architecture



Takeaway: config-driven orchestration with a shared dataset contract and registry-based extension.

YAML Overview

The pipeline begins as a compact YAML specification.

- Names the loader
- Selects the split policy
- Lists pre- and post-split transforms
- Keeps the workflow readable in one place

```yaml
loader:
  type: mock_calibrator_npz
  params:
    path: examples/mock_calibrator/src/toy_timeseries.npz

pre_split:
  - name: fill_nan
    params:
      value: 0.0

split:
  policy: chronological
  params:
    ratios: [0.7, 0.15, 0.15]

post_split:
  - name: repair_rolling_outliers
    apply: [train, val]
    params:
      window: 25
      sigma: 3.0
      min_periods: 5

  - name: standard_scale
    fit: train
    apply: [train, val, test]
    state_key: standard_scale
```

PipelineConfig Overview

YAML is parsed into one validated typed object for the pipeline.

- Groups loader, split, and transform settings
- Carries metadata alongside execution settings
- Validates the config before runtime

```
@dataclass
class PipelineConfig:
    """Validated configuration for one preprocessing pipeline.

    This dataclass is the typed representation of a parsed YAML
    preprocessing config. It records the loader, split policy,
      optional
    transform stages, and user metadata consumed by the
      preprocessing
    pipeline.
```

PreprocessingPipeline Overview

The orchestrator resolves named components and runs the workflow in order.

- Resolve loader, split, and transforms
- Load once
- Apply pre-split transforms
- Split, then apply post-split transforms
- Return a dataset bundle

```
@dataclass
class PreprocessingPipeline:
    """Orchestrator for one configured preprocessing build.

    A pipeline loads the full dataset once, applies full-dataset
      transforms,
    computes the train/validation/test split indices, materializes
      split
    datasets, and then applies split-specific transforms in
      deterministic
    ‘‘train‘‘, ‘‘val‘‘, ‘‘test‘‘ order. Fitted post-split
      transform state is
    retained during the build and copied into the returned
      DatasetBundle.

    Attributes
```

Custom Registry Overview

Public registries let private repositories plug in project-specific code.

- Register loaders, transforms, and split policies
- YAML refers to them by short names
- Same seam works for built-ins and private extensions

```
def register_loader(name: str, fn: Loader, *, replace: bool =
      False) -> None:
    """Register a loader under a short config name."""
    _register(LOADERS, kind="loader", name=name, fn=fn,
      replace=replace)


def register_transform(name: str, fn: Transform, *, replace:
      bool = False) -> None:
    """Register a transform under a short config name."""
    _register(TRANSFORMS, kind="transform", name=name, fn=fn,
      replace=replace)


def register_split_policy(name: str, fn: SplitPolicy, *,
      replace: bool = False) -> None:
    """Register a split policy under a short config name."""
```

BaseDataset Overview

Every loader, transform, and split works on the same canonical in-memory object.

- Shared dataset contract across the framework
- Holds aligned arrays and metadata
- Exposes validation and split tagging helpers

```
@dataclass
class BaseDataset:
    """Canonical container for one full or split time-series
      dataset.

    ‘‘BaseDataset‘‘ stores data in ‘‘(T, C, F)‘‘ form, where
      ‘‘T‘‘ is the
    number of time rows, ‘‘C‘‘ is the number of channels, and
      ‘‘F‘‘ is the
    number of features. Optional arrays and names are validated
      against these
    dimensions so loaders, transforms, and split policies can
      share a common
    dataset contract.
```

DatasetBundle Overview

The pipeline returns one structured object for downstream use.

- Makes train, val, and test explicit
- Carries shared metadata for the build
- Clean handoff to modeling or analysis code

```
@dataclass
class DatasetBundle:
    """Container for the canonical train, validation, and
      test datasets.

    ‘‘DatasetBundle‘‘ is the final output of a
      preprocessing build. It groups
    the three split datasets with metadata that applies to
      the bundle as a
    whole, such as fitted transform state recorded by the
      pipeline.
```

Example Dataset: Mock Calibrator

A concrete example shows how project-specific data plugs into the public preprocessing core.

- Synthetic multichannel time-series dataset used for demos and tests
- 10,000 timesteps, 8 channels, 4 features per sample
- Features include clean signal, noise, noisy signal, and SNR
- Injected low-SNR anomaly periods create outliers for repair and scaling

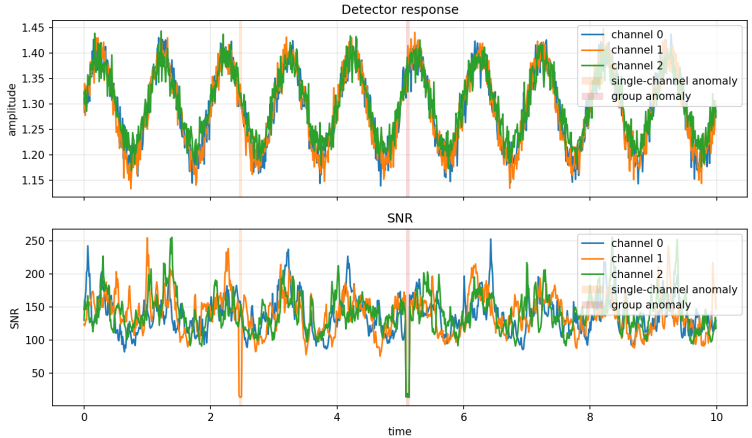
The example keeps the core generic:

- custom loader registered as `mock_calibrator_npz`
- standard YAML pipeline drives the run
- output is the same core `DatasetBundle` contract

Generator settings used here:

- `dt = 0.01`, seed `123`
- plotted channels: `[0, 1, 2]`
- plotted range: first `1000` timesteps

Mock Calibrator: Example Plots



Top: noisy signal. Bottom: SNR with highlighted single-channel and group anomaly periods.

Mock Calibrator: Input YAML

The example pipeline is still just declarative configuration.

- selects the custom loader
- fills NaNs before splitting
- splits chronologically
- repairs rolling outliers on train and validation
- fits scaling on train and applies it to all splits

```yaml
loader:
  type: mock_calibrator_npz
  params:
    path: examples/mock_calibrator/src/toy_timeseries.npz

pre_split:
  - name: fill_nan
    params:
      value: 0.0

split:
  policy: chronological
  params:
    ratios: [0.7, 0.15, 0.15]

post_split:
  - name: repair_rolling_outliers
    apply: [train, val]
    params:
      window: 25
      sigma: 3.0
      min_periods: 5

  - name: standard_scale
    fit: train
    apply: [train, val, test]
    state_key: standard_scale
```

Mock Calibrator: Run Summary

Running the example uses the same generic experiment entry point as any other dataset.

- setup module registers the custom loader
- built-in transforms and split policy are reused unchanged
- successful run returns train, validation, and test tensors

```
$ python scripts/run_experiment.py \
    --config examples/mock_calibrator/pipeline.yaml \
    --setup examples.mock_calibrator.register

train: (7000, 8, 4)
val:   (1500, 8, 4)
test:  (1500, 8, 4)
```

Conclusion

The preprocessing core provides a compact, reusable path from declarative configuration to a reproducible in-memory `DatasetBundle`.

- YAML defines the workflow without embedding pipeline logic in notebooks or scripts
- Typed config and pipeline classes keep execution consistent and inspectable
- Public registries let private repositories add loaders, transforms, and splitters cleanly
- The result is a standard `DatasetBundle` ready for downstream modeling

Backup: Code Size

```
$ cloc src/
      15 text files.
      15 unique files.
       0 files ignored.

github.com/AlDanial/cloc v 2.04  T=0.02 s (739.9 files/s, 65407.3 lines/s)
-------------------------------------------------------------------------------
Language                     files          blank        comment           code
-------------------------------------------------------------------------------
Python                          14            244            219            824
Markdown                         1             11              0             28
-------------------------------------------------------------------------------
SUM:                            15            255            219            852
-------------------------------------------------------------------------------
```

Note: the full Python implementation in `src/` is only **824 lines of code**.